

LEGACYLABS

Agentic Domain-Driven Modernization

Where the Agent Ends and the Architect Begins

Pierre Milet Llobet

PROJECT ROSETTA · EXPLORE

About this book

This is not a book about tools. The tools it mentions, a compiler that rewrites an old language into a new one, a framework that organizes code into vertical slices, a graph that projects a system onto a navigable structure, an agent that reads, reasons and writes, will all age. Some will be obsolete before you finish reading. That is why they always appear as examples of an idea, never as the idea itself. What the book defends is a method, and a method is measured by the principles that hold it up, not by the parts that happen to embody it today.

The book has one centre and two axes. The centre is the domain. The two axes are borrowed. The first is Gregor Hohpe's architect's elevator, which runs from the penthouse, where strategy is decided, to the engine room, where code executes. The second is Kent Beck's 3X model, Explore, Expand, Extract, which describes how an idea matures from a hunch into a standard. This book lives, declaredly, in the Explore phase. It does not sell you a method proven against hundreds of production migrations. It offers you an argument, coherent and reasoned, for why a method ought to work, and it invites you to walk it in fast learning cycles to discover whether it deserves to move to Expand.

That honesty is deliberate. A book that overclaims is refuted by a single failed engagement. A book that argues from principles endures, because principles are not refuted by anecdotes. If five years from now the tools of today are archaeology but the argument still stands, the book will have done its job.

One last note. The examples are generic. There is no single concrete system to return to chapter after chapter, because tying the method to one case ties it to that case's accidents. When you need to ground an idea, picture the ugliest inherited system you know. It will do.

Contents

Prologue. The Modernization That Does Not Modernize

Part I. Why the Domain, Not the Code 1. The Relocation Trap 2. The Legacy System as Oracle 3. Fidelity to the Invariant, Not the Structure

Part II. The Agent on Every Floor 4. The Architect's Elevator, Now with Agents 5. Where the Agent Ends and the Architect Begins 6. The Discipline of Provenance

Part III. Explore Before You Productize 7. The 3X Model Applied to Modernization 8. Recovering the Domain 9. From the Recovered Domain to the Target Architecture

Part IV. Proving You Did Not Break It 10. The Transitive Chain 11. Behaviour as an Executable Specification 12. Black-Box Parity and the Law of Order

Part V. Living with Two Systems 13. The Strangler, Honestly 14. Cutover Without Faith 15. Retiring the Oracle Without Losing the Reference

Part VI. From Explore to Expand 16. This Is a Hypothesis

Appendices - A. Antipatterns - B. Glossary

Prologue. The Modernization That Does Not Modernize

Imagine you inherit a system. It has been in production for decades. It moves money, or lives, or freight, something that matters. Nobody who wrote it still works at the company. The documentation, if it exists, lies. And yet the system works, day after day, with a reliability no recent project has matched. Your job is to modernize it.

The first temptation is the most reasonable and the most wrong. It consists of treating the problem as a translation. You have code in an old language, you want code in a new one, so you translate. There are tools today that do it almost on their own. A transpiler takes the source language and emits the target one, line by line, structure by structure. The result compiles. It passes the tests, if there are any. And you have failed, though it will take months to notice.

You have failed because you confused the symptom with the disease. The problem with a legacy system is not that it is written in an old language. The problem is that its design, the decisions about which concept lives where, which rule protects which data, which part knows about which other part, was made decades ago, eroded with every patch, and was never looked at as a whole again. That design is the debt. Translating the language keeps the debt intact. You end up with the same tangle, now in modern syntax, and you have spent a budget moving it from one place to another. The syntax is new. The design is the same as ever. That is not modernization. It is relocation with better handwriting.

There is a more sophisticated variant that falls into the same trap. Instead of translating, you recompile. You take the conceptual binary of the old system and run it on a new platform without touching the source code. It is cheaper, faster, less risky in the short term. And it is still not modernization, because the design has not changed. You have changed the floor under the system's feet, not the system. It is a legitimate relocation tactic, sometimes the right one, but calling it modernization is lying about what you have bought.

This book starts from an uncomfortable claim. **Modernization is a domain problem, not a code problem.** The code is the way the domain was once expressed, under constraints that no longer exist. To modernize for real is to recover the domain the code hides, understand it, and express it again in an architecture that serves today's constraints. The target language is the easy part and the least important. The work is in the domain.

If you accept that, the rest of the book follows. If modernization is a domain problem, you need the conceptual tools of the domain, and the most mature one we have is Domain-Driven Design. You need to recover the model hidden in a system nobody fully

understands, which requires interrogating the system itself rather than trusting its code or its documentation. You need to redesign without breaking, which requires a verification discipline that proves equivalence rather than assuming it. And you need to live alongside the old system while the new one is born, because no system that matters is switched off all at once.

There is a second thread in this book, and it enters now. As I write this, a new class of tool has begun to change the economics of everything above. Agents, programs that read, reason about what they read, and act, can traverse a legacy system at a speed and scale that were impossible before. But agents do not change the method. They change what the method can afford. A domain recovery that used to cost months of a scarce expert can now cost days, which means you can afford to explore, get it wrong, and redo it. Tasks that used to belong to the Extract phase, mechanical and expensive, become Explore again, cheap and reversible. That is the real news. Not that agents write code, but that they make learning so cheap that they change which phases of the work you can walk.

That is why this is a book about agentic DDD applied to legacy modernization. The domain at the centre. Agents on every floor of the elevator, from the penthouse to the engine room, augmenting the architect at each level. And one question that runs through the whole text and gives it its subtitle. Where the agent ends and the architect begins. That boundary, you will see, is not a matter of convenience or trust. It is an architectural decision, perhaps the most important you will make.

Let us start at the beginning, which is to understand well why translation fails and what replaces it.

Part I. Why the Domain, Not the Code

1. The Relocation Trap

It is worth looking closely at why translation fails, because the failure is subtle and seductive. A transpiler does exactly what it promises. It takes a structure in the source language and produces its equivalent in the target. A loop becomes a loop. A condition becomes a condition. A routine becomes a method. The mapping is faithful, and there lies the problem. Structural fidelity is precisely what you do not want to preserve.

Think about what is inside a typical legacy program. There is business logic, the rules that define what the system means, why an amount is rejected, when an operation is valid. And there is mechanism, the plumbing that made that logic work on its era's platform, how a file is opened, how a block of memory is passed from one program to another, how an input-output error is handled. In the old system, logic and mechanism are interwoven, because the platform of the time forced them to be. There was no other way.

When you translate line by line, you translate the two together. The old mechanism, which only existed to satisfy a platform you are abandoning, travels to the new system disguised as logic. You end up with modern code that opens files as if it were still on the old platform, that passes blocks of memory between modules as if memory were scarce, that handles errors with numeric codes that no longer mean anything. You have imported the plumbing of a house you demolished. And because it is now in modern syntax, it is harder to detect, because it looks like legitimate new code.

The design suffers the same fate. If the old system put three distinct responsibilities in the same module because creating modules was expensive, the translation produces a new module with the three responsibilities. If two concepts that should be separate shared a data structure to save space, they keep sharing it. The wrong boundaries are preserved with perfect fidelity. The coupling is conserved. The missing cohesion stays missing. The only thing that has changed is the colour of the syntax.

This is where many well-meaning people defend themselves. They say, fine, we translate first and refactor later. First a faithful equivalent, then we clean it up. It sounds prudent. It is a deeper trap, for two reasons.

The first is economic and psychological. Once you have a new system that compiles and works, the budget for the cleanup disappears. The project is declared finished. The translated tangle stays, now blessed as a modern codebase, and the next ten years of maintenance will treat it as sacred. The refactor that was going to come later never comes. What was meant to be an intermediate step becomes the destination.

The second reason is more technical and more interesting. Refactoring requires knowing which behaviour you must preserve while you change the structure. If you translated without understanding the domain, you do not have that knowledge. You do not know which of those three responsibilities of the module is business logic that must survive and which is mechanism that must die. You have no safety net to tell you whether your cleanup has broken anything. Refactoring blind over code you do not understand is not modernization, it is roulette. We will come back to this carefully, because the safety net, the proof that you have not broken anything, is one of the pillars of the method.

Recompilation, the other route, is honest in its own way and therefore less dangerous, provided you do not lie about it. Running the old system on a new platform without touching the code is a relocation. It has legitimate uses. It gets you off dying hardware, removes a dependency on a vendor, buys you time. What it does not do is modernize, because the domain is still expressed exactly the same, with the same wrong boundaries and the same interwoven mechanism. The error is not recompiling. The error is believing you are done.

There is a principle worth fixing now, because it will reappear. **Changing the platform and changing the design are two distinct jumps, and they must never be made at once.** If you recompile to change platform, do it as a clean, isolated move, without touching the design. If you redesign, do it on a stable platform, without moving it under your feet. Whoever attempts both jumps in the same move is left without a foothold to diagnose what failed when something fails, and something will fail. This forbidden double jump is one of the most expensive and most common errors, and it has its own chapter later.

So, if it is neither translating nor recompiling, what is modernizing. The short answer is to recover the domain and express it again. The long answer takes the rest of the book. But there is a prior turn, almost a change of attitude, without which none of the rest works. You have to stop seeing the old system as a liability to escape from and start seeing it as the most reliable source of truth you have. That is the next chapter.

2. The Legacy System as Oracle

The legacy system has a bad reputation, and it has partly earned it. It is rigid, opaque, expensive to maintain, impossible to hire people who know it. But there is one thing that system does better than any document, any expert, and any specification you could write about it. It knows exactly what it does. It proves it every day, in production, without getting wrong the rare cases you do not even know exist.

That is the idea that changes everything. **The old system is not just a liability. It is an oracle.** An oracle, in the sense we give it here, is a source that answers a question about behaviour with truth. You give it an input, it gives you the correct output, correct by definition, because it is what the system actually does and what the business has accepted as correct for decades. You do not have to believe anyone about how a surcharge is calculated in the limit case, or what happens when two operations arrive in the improbable order. You ask the system and it tells you.

This reorders the priorities of recovery. Tradition says that to understand an old system you read its code and its documentation. The code is the better of the two sources, but it is treacherous, because it mixes logic and mechanism, because it is full of dead paths that never run any more, and because reading it tells you what the system appears to do, not what it does. The documentation is worse, because it describes what someone wanted the system to do at a given moment, and the system has been diverging from that intention for years without anyone updating the document. The two sources give you hypotheses. The oracle gives you facts.

The practical change is enormous. Instead of arguing for weeks about what an unintelligible routine means, you run it with chosen inputs and observe. Instead of trusting that an expert remembers the special-case rule well, you generate the special case and look at the answer. The old system becomes a laboratory. You run experiments on it. The questions that used to produce meetings now produce observations.

For the oracle to be truly useful it needs two things, and this is where the tools come in, as examples of a concept. The first is that it answers fast. An oracle that takes a night to give you an answer is no good for exploring, because exploration needs short cycles, ask, observe, adjust the question, ask again, dozens of times a day. That is why you want a copy of the old system that runs locally, in milliseconds, not on the production mainframe with its job queue. One way to get there is to recompile the old system onto a modern platform you can run on your own machine, in a container, as often as you like. The compiler that does that recompilation is an example of a tool. The durable concept is the fast, local oracle, a faithful replica of the old behaviour that answers in the time of a thought cycle.

Notice something elegant. Recompilation, which in the previous chapter was a false modernization if you confused it with the destination, is here exactly the right tool for a different purpose. You are not recompiling to ship a system. You are recompiling to build yourself an oracle. The same technique, despised as a goal, is valuable as an instrument. The difference is what you use it for.

The second thing the oracle needs is fidelity. It has to behave like the original, not resemble it. If your replica diverges from the real system in the rare cases, it will be lying to you precisely where you will most rely on it. That is why the fidelity of the replica is not assumed, it is validated, by comparing it with the real system over enough cases. An unverified oracle is a rumour with a good reputation.

There is an honest caveat worth not hiding. The oracle only knows what it is asked. It knows the behaviour over the inputs you give it, not over the ones you do not think of. If a path of the old system never runs in your experiments nor in the real traffic you observe, its behaviour stays uncaptured, and it is behaviour that may matter on the least expected day. The oracle does not remove uncertainty, it bounds it and makes it visible. This will have consequences when we talk about which inputs to choose to interrogate it, because choosing the questions well is the difference between an oracle that covers the boundaries where the bugs live and one that only confirms what already happens every day.

For now, keep the change of attitude. The system you were asked to replace is, while it lives, your best teacher. Before you retire it you have to learn from it everything you will need forever, because when it is switched off, that source of truth disappears. Much of the method consists of squeezing the oracle with discipline while it is still switched on.

3. Fidelity to the Invariant, Not the Structure

If the old system is the oracle and what you are after is preserving its behaviour, a tricky question arises. Preserve what, exactly. Because if the answer were to preserve everything, modernization would be impossible, you would be forced to keep down to the last detail of the old system, including the tangle you wanted to remove. Fidelity misunderstood takes you back to the translation trap.

The right answer requires a distinction that is the conceptual heart of this chapter and one of the keys to the whole method. There are two very different things an old system contains, and only one of them is sacred.

The first is the what. The business rules, the invariants that must always hold, the semantics of each operation. An amount can never exceed a certain limit. An operation on an expired entity is rejected. A certain calculation rounds in a certain way and the result matters to the last decimal because it is money. This is the problem space. It exists independently of how the old system decided to implement it. It would survive even if you rewrote the system from scratch in any language. This is what you must preserve with absolute fidelity.

The second is the how. The concrete structure the old system chose to satisfy those rules. Which module contained which logic, which data structure which concepts shared, in what order things were done, where the boundaries were drawn. This is solution space, and it is an old solution, taken under constraints that no longer exist. It is not sacred. It is, in fact, exactly what you came to change.

Here is the sentence worth engraving. **Fidelity is to the invariants and the semantics, not to the structure.** You preserve what the system means. You are free to redesign how it achieves it. That freedom is not a minor permission, it is the entire prize of modernization. Without it you would be condemned to copy the old design. With it you can redraw boundaries, separate what was mixed, give each concept its place, and still guarantee the system keeps meaning the same thing.

This distinction has a consequence that seems surprising and that will later be a technical pillar. If what you preserve is the semantics and not the structure, then two systems can be equivalent in behaviour and completely different inside. The new system can have other boundaries, other modules, another order, and still give the same answer to the same question. That means the proof that you have not broken anything cannot consist of comparing the innards of the two systems, because by design they are different. It has to consist of comparing what is observed from the outside, the behaviour, not the implementation. That is the seed of black-box parity, which is what we will call verifying equivalence by looking only at observable inputs and outputs, never the inside. We will develop it fully in Part IV.

The distinction also lights up where the most dangerous errors of a modernization hide. There are pieces of an old system that look like mechanism but carry a business decision inside. The classic example is the handling of a technical error. When the old system does not find a piece of data and reacts in a certain way, is that reaction pure plumbing, detecting an absence, or is it a business rule in disguise, deciding that an absence equals a certain default value with real consequences? If you treat it as mechanism and discard it, you have silently lost a rule. If you treat it as a rule and keep it, you drag along plumbing. Telling which is which cannot be done by reading alone, because the code interweaves the two things. It is done by asking the oracle what business consequence that behaviour has. The silent loss of semantics, keeping the technical shell and losing the rule it hid, is one of the most expensive antipatterns, and it appears in its appendix.

It is worth closing with an image that sums up the three ideas of this first part. Think of the recovered domain as a model you will be able to dress in many different architectures. The model, the rules and the semantics, is one and is stable. The possible architectures over that model are many. Translation fails because it copies an old architecture without recovering the model. The oracle lets you recover the model with facts instead of rumours. And fidelity well understood tells you which of the model is untouchable, the semantics, and which is free, the structure. With that in hand, the question stops being how do I translate this code and becomes which of the many possible architectures over this domain best serves today's constraints. That is an architect's question. And architects, it turns out, now work with company.

Part II. The Agent on Every Floor

4. The Architect's Elevator, Now with Agents

Gregor Hohpe proposed a metaphor that helps in understanding what an architect does, and which here we are going to stretch a little beyond its original purpose. Picture an organization as a building. In the penthouse, strategy is decided, where the business is heading, what matters. In the engine room, many floors below, code executes and things actually happen. In between there are intermediate floors, the architecture, the teams, the processes. Most people live on a single floor. The penthouse crowd talk about vision and do not know what is in the binaries. The engine-room crowd know every bit and do not know why it matters. The value of the architect, Hohpe says, is that they ride the elevator. They go up and down. They translate strategy into structure and structure into strategy. They connect floors that would otherwise not speak to each other.

In a modernization, the elevator is literal. In the penthouse are the questions of the domain, what concepts this business has, what rules govern it, where the natural boundaries between its parts lie. In the engine room is the old system, its code, its data structures, its interwoven routines. And the work of modernizing is precisely an elevator journey, go down to the engine room to recover what the system does, go up to the penthouse to understand what that means in domain terms, go back down to verify, go up to decide the architecture. Whoever can only read code cannot modernize, because they do not see the domain. Whoever only knows the domain cannot either, because they cannot recover anything from a system they cannot read. Modernization is an elevator trade.

Here the book's second thread enters, and it is worth saying with precision so it does not age. A class of tool has appeared, agents, that can ride that elevator. An agent, in the sense that matters here, is a system capable of reading a great deal of material, reasoning about it, and producing results, code, models, analyses, proposals. It is not magic and it is not a colleague. It is a tool with a new and important property. It operates on every floor. It can read code in the engine room. It can propose domain boundaries in the penthouse. It can generate the scaffolding of an architecture on the intermediate floors. Before, each floor required a different tool and a different person. Now there is a class of tool that rides them all.

This is what justifies the title's adjective, agentic, and it is worth understanding well why. It is not that agents are a one-off accelerator of a concrete task, like a faster compiler. It is that agents are present at every level of the work, augmenting the architect on every floor of the elevator at once. The architect who used to go down to the engine room alone now goes down with a tool that reads ten times faster and does not tire. The one who used to go up to the penthouse to propose a context map now goes up with a tool that has already read the whole system and brings candid-

ates. DDD stays at the centre, intact, because DDD is the discipline of putting the domain first, and that was not invented by agents nor will it be replaced by them. What changes is that the domain is now recovered, modelled and verified with a companion on every floor.

Two opposite errors about this must be resisted. The first is disdain, believing that agents are autocomplete with pretensions and that the method changes in nothing. The second is fascination, believing that agents do the work and the architect is redundant. The two errors fail in the same place, they do not understand what exactly changes. What changes is not the principles, it is the economics. And when the economics of an activity change enough, they change which activities you can afford, and that does alter the method, not in its principles but in its reach.

The clearest example is given by Beck's own 3X model, which we will see in detail later. Beck distinguishes three phases in the life of an idea. Explore, when you look for what works and almost everything you try is discarded. Expand, when something works and you have to scale it. Extract, when it is already stable and you squeeze its efficiency. In a traditional modernization, recovering the domain was an Extract task, expensive, slow, done by scarce experts, so you did it once, grudgingly, and called it good even when it was bad, because redoing it was unthinkable. With an agent that recovers in days what used to cost months, that same task drops in cost until it becomes Explore. You can afford to recover the domain, look at it, not be convinced, throw it away, and recover it again another way. What was an irreversible commitment becomes a cheap experiment. That is the deep transformation. Not that the agent recovers the domain, but that it makes recovering it so cheap that recovery stops being a single solemn act and becomes a fast learning cycle.

So the agent rides the elevator with the architect, on every floor, and cheapens the journey to the point of changing which journeys you can afford. But there is a floor where the agent does not decide, it only proposes. Knowing which boundary that is, and why it has to exist, is the next chapter, and it is the most important decision in the book.

5. Where the Agent Ends and the Architect Begins

If the agent rides every floor, the natural and dangerous question arises. So, what does the architect do that the agent does not. The lazy answer is whatever the agent cannot do yet, and it is lazy because it ages with every new version of the agent and because it treats the boundary as a temporary technical limitation. The right answer is different and does not age. There is a class of decision the architect must make not because the agent cannot, but because it must not be delegated, and understanding why is understanding the method.

Let us start with how an agent reasons, because that is where the key lies. An agent does not reason well over raw source code. Code is long, repetitive, full of irrelevant detail, and it interweaves logic with mechanism. What an agent does well is reason over projections of the system, representations that distil one dimension of the system and discard the noise. A graph that shows what calls what. A model that shows what data flows toward what decision. A structure that shows what rules protect what entity. Over those projections the agent reasons with power, because the projection has already done the work of separating signal from noise. The code is the territory. The projection is the map. And agents, like explorers, work better with maps.

This already draws a first boundary. Building good projections, deciding which dimension of the system deserves a map, is the architect's work. The agent navigates the map, but which map to draw for which question is a design decision. A wrong map leads a most powerful agent to a wrong conclusion with total confidence, which is the worst kind of error.

But the underlying boundary is deeper and has to do with the difference between finding and choosing. When you recover a domain from an old system, there are things you discover and things you decide. You discover that a certain rule exists, because it is there, in the oracle's behaviour, and you can point at it. That is a finding. It has a source. An agent can do it, because it consists of observing and reporting what is observed. By contrast, when you decide that two concepts the old system mixed should go in separate bounded contexts, you have not found that anywhere. The old system did not tell you. You chose it, in the light of your judgement about what best serves today's business. That is a design decision. It has no source in the old system, because its source is you.

The boundary between the agent and the architect is the boundary between finding and choosing. The agent finds. It reports what the system does, with its source, because finding is observing. The architect chooses. They decide which architecture, which boundaries, which rules to consolidate or separate, because choosing is an act of judgement that does not derive from any observation. And the reason the

boundary must not be crossed is not that the agent is incapable of proposing an architecture, of course it can propose one. The reason is that a proposal with no source, presented as if it were a finding, contaminates the model with decisions disguised as facts. And a model where you cannot tell what you recovered from the system from what you invented is a model you cannot trust for anything important, because you do not know how much of it is truth about the business and how much is a tool's opinion.

This turns the boundary into an architectural decision, not one of convenience. You do not say the agent does up to here because beyond it gets things wrong. You say the agent finds and the architect chooses because mixing the two destroys the reliability of the model. It is an epistemic boundary, about what kind of knowledge each thing is, and it turns out to coincide exactly with the boundary of who does what. The recovered, which has a source, the agent can bring. The decided, which has no source other than judgement, the architect must put there, and it must be marked as such.

It is worth insisting on an asymmetry people underrate. The agent can propose design options, and it should, because it is very useful there. The right thing is for the agent, after recovering the domain, to present the architect with a fan of possible architectures, each one with its evidence and its consequences, especially its consequences for the later verification. This option is cheaper to test but consolidates two rules into one. This other is more faithful but more expensive to verify. The agent is excellent at setting that table of options. What it cannot do is sit at the table and choose for you, because choosing is committing judgement, and judgement answers to the business, not to a tool. The agent serves the menu. The architect orders.

A practical question remains. If the recovered and the decided have to be distinguished, how are they distinguished in practice, within the model, so that anyone who reads it knows what they are looking at. The answer is a simple and rigorous discipline, and it is the next chapter.

6. The Discipline of Provenance

A domain model recovered from an old system is a mixture of things from very different origins. Some you extracted directly from the observed behaviour. Others you inferred from the structure, they seem true but you have not confirmed them. Others a business expert who has been with the system for thirty years confirmed for you. Others are design decisions you made yourself. And others are simply conjectures you have not yet tested. They all coexist in the same model, and from the outside they look alike, sentences that describe the domain. That appearance of sameness is a danger, because it invites you to trust them all equally, when they deserve very different degrees of trust.

The solution is a discipline I ask you to take seriously even though it sounds bureaucratic, because it is what holds up the reliability of the whole method. **No statement enters the model without declaring its provenance.** Every rule, every boundary, every invariant you write carries a mark that says where it comes from and, therefore, how much you can trust it.

The categories of provenance are few and clear. There are extracted statements, observed directly in the oracle's behaviour, the most reliable, because you have seen them happen. There are statements inferred from structure, deduced from how the system is organized, plausible but unconfirmed, because the structure can deceive. There are statements inferred from the name, suggested by what something is called, the weakest, because names in old systems lie frequently. There are statements attested by an expert, which a human who knows the business confirms. And there are unproven statements, conjectures honestly marked as such so that nobody takes them for facts. The concrete marks you use do not matter, the distinctions they capture do.

There are a couple of subtleties worth pausing on, because that is where the discipline proves its worth. The first. What is attested by an expert is not the same as what is extracted from the oracle, even though both seem firm. The expert may remember wrong, may describe how they believe the system works rather than how it works, may be describing a version from ten years ago. Their testimony is valuable, but it is testimony, not observation. That is why it carries its own mark, distinct from the extracted. When the expert's testimony and the oracle's observation disagree, the discipline of provenance warns you that you have a conflict to resolve, instead of letting you choose blind. And almost always the oracle wins, because the system does what it does, whatever the memory says.

The second subtlety connects this chapter with the previous one and closes the idea of the boundary. There is a golden rule. **What can cite no source is not a finding,**

it is a choice, and it must leave the model of the recovered for a register of decisions. If you write a statement about the domain and on trying to mark its provenance you discover you cannot point at where it comes from, that you did not extract it nor infer it nor have anyone attest it, then you are not describing the old system. You are deciding something about the new one. That statement does not belong in the model of the recovered. It belongs in the register of design decisions, which is another matter entirely, the input to the redesign, not part of the portrait of the domain.

This rule is exactly the boundary of the previous chapter, finding versus choosing, made operational. Provenance is the mechanism by which the boundary between the agent and the architect stops being an intention and becomes an auditable practice. Anyone can take the model and ask, of each statement, where it comes from. If everything recovered has a source and everything that has no source is in the register of decisions, the boundary is clean. If you find statements with no source camouflaged among the recovered ones, you know that someone, a human or an agent, has slipped a choice disguised as a finding, and you know the model is contaminated right there.

Some will say this is too much ceremony for a piece of engineering work. The answer is that the cost of the ceremony is small and the cost of not having it is enormous and deferred. Without the discipline of provenance, six months later nobody remembers whether the limit-case rule was observed in the old system or invented by someone in a meeting, and that difference is the difference between a fact about the business and an opinion that might be wrong. Provenance is cheap to maintain if you maintain it from the start, and ruinously expensive to reconstruct if you did not. It is, at bottom, the same lesson as the verification we will see later. Better to prove than to assume, and better to know where each thing comes from than to trust that someone remembers.

With the boundary drawn and provenance in place, we now have the two halves of the agentic method, the agent that finds and the architect that chooses, separated by a clean and auditable line. Now we can move to the method proper, which begins with a question of attitude, when it is prudent to productize what you are learning and when it is not. That is where Beck comes in.

Part III. Explore Before You Productize

7. The 3X Model Applied to Modernization

Kent Beck observed that software lives three very distinct phases, and that confusing them is one of the most common causes of disaster. He called them Explore, Expand, Extract, the three Xs. In Explore you look for what works, and the uncomfortable truth of this phase is that most of what you try does not work, so the value lies in trying cheaply, learning fast, and discarding without grief. In Expand something has worked and suddenly you have to grow to serve a demand that is spiking, and the problem stops being what to do and becomes holding up. In Extract you already know what you do and you do it at stable scale, so the game is to squeeze efficiency, polish, optimize. Each phase calls for a different mindset, and applying the mindset of one to another is the error. Optimizing in Explore is polishing something you are going to throw away. Exploring in Extract is introducing chaos where stability was needed.

The thesis of this chapter, and one of the most important attitude decisions in the book, is that **a young modernization methodology is in Explore, and treating it as if it were in Extract is the fastest road to failure**. This applies at two levels, and it is worth separating them because it is easy to mix them up.

The first level is the method itself. This book describes a method that is in its Explore phase. It has not been validated against hundreds of production migrations. It is a reasoned hypothesis, an argument for why it ought to work. Treating it as a mature process, codifying it into a rigid corporate methodology and deploying it at scale before having walked it in the small, would be committing Beck's error in its purest form, productizing something you are still exploring. The right attitude is the opposite. Walk the method in small, fast cycles, over bounded pieces, learning from each cycle, discarding what does not work, until the evidence justifies moving it to Expand. The method earns the right to scale, it does not arrogate it.

The second level is each concrete modernization engagement. A modernization also starts in Explore, even if organizational pressure pushes you to pretend it starts in Extract. The pressure says, we already know what to do, let us plan the eighteen phases and execute. It is a lie, and it is an expensive lie. At the start of an engagement you do not know enough. You do not know the real domain hidden in the system, you do not know where the natural boundaries lie, you do not know which parts are logic and which are mechanism, you do not know where the rare cases that will bite you are. Pretending a detailed plan over that ignorance produces a plan of fiction that reality destroys in the first month.

The alternative is to start with what deserves to be called an architecture spike, borrowing the term from agile practice. A spike is a time-boxed experiment whose goal is not to produce a deliverable but to produce knowledge. You take a small, repres-

entative slice of the system, ideally one that touches the most feared parts, and you walk it end to end, recovery, redesign, verification, all at minimal scale. Not to ship it, but to learn. At the end of the spike you know things no prior plan could have given you. You know how much it really costs to recover the domain of this system. You know whether your oracle is faithful. You know where the rare cases are. You know which boundaries make sense. With that knowledge, the plan you write afterwards is a real plan, not a fictional one.

This is where the agentic cheapening of chapter four changes the rules in a deep way. In a world without agents, spikes are expensive, because recovering domain by hand is slow, so the temptation to skip them and plan blind is strong, and often it wins. In a world with agents, the spike is cheap, because the recovery that used to take weeks takes days. And when the spike is cheap, you can afford several, exploring several slices, several boundary hypotheses, several candidate architectures, before committing to any. What used to be a luxury almost nobody could afford, exploring for real before deciding, becomes the default option. The agent does not only speed up the work. It makes prudence affordable.

There is a discipline that accompanies 3X and is worth declaring without circumlocution. Do not productize prematurely. The temptation, when a spike goes well, is to declare victory, freeze the approach and apply it en masse. It is the temptation to jump from Explore to Extract without passing through Expand, to turn a successful experiment into a standard before having tested it on several distinct cases. A single successful spike does not prove the method works, it proves it worked once, which is much, much less. The sign that you are ready for Expand is not that something worked, it is that you understand why it worked well enough to predict when it will not. Until you have that, you are still in Explore, and it is fine to be in Explore. It is where you learn.

This attitude, walking in fast cycles before committing, fits exactly with the elevator of chapter four. Each cycle is a complete elevator journey, you go down to recover, up to model, you decide, down to verify. The difference between doing it well and badly is not in the floors, which are the same, but in the speed and the humility of the journey. Fast, to learn a lot per unit of time. Humble, to throw away without grief whatever the journey reveals does not serve. With that attitude in place, the first real journey is to go down to the engine room to recover the domain. That is the next chapter.

8. Recovering the Domain

Recovering the domain is the central act of modernization, and it is where the agentic domain-driven method is won or lost. The goal is to produce a faithful portrait of the domain the old system embodies, expressed in the terms of Domain-Driven Design, stripped of mechanism, and with each statement marked by its provenance. Call that portrait the recovered model. It is not the old system and it is not the new one. It is the domain, the what, separated from the old how and still not committed to any new how.

It is worth being very precise about what this model is and is not, because confusion here poisons everything else. The recovered model describes the current state of the domain, not the desired one. It is a portrait of the as-is, not the to-be. This surprises whoever expects modernization to start by drawing the ideal system. No. It starts by understanding with precision what is there, because you cannot preserve the semantics of something you have not understood, and preserving semantics is the contract of fidelity from chapter three. The ideal system comes later, and it comes as a design decision, marked as such, not as part of the portrait.

The recovered model has two layers, the two classic ones of DDD. The tactical layer describes the inside of each part, the entities and their identities, the aggregates and the invariants they protect, the value objects, the relationships. The strategic layer describes the whole, the context map, the boundaries between the distinct parts of the domain and how they relate. Recovering both is the work. The tactical tells you which rules live inside each concept. The strategic tells you where the seams of the domain run, which is where you will later be able to cut.

Here there is a nuance many people overlook and it is worth nailing down. The recovered model is not agnostic to the solution space. It contains solution space, but the old one. The aggregate and context boundaries you recover are design decisions, only decisions someone else made decades ago. You recover them because they are valuable information about how the system understood its own domain, not because they are sacred. Hence the model has two distinct levels of fidelity, and distinguishing them is crucial. There is a mandatory layer, the invariants and the semantics, which is problem space and which survives any redesign, because it is the what. And there is an informative layer, the recovered structure, the old boundaries, which is old solution space, valuable as information but re-decidable, because it is a how you came to change. Confusing the two layers is the error. If you treat the recovered structure as sacred, you go back to the translation trap. If you treat the invariants as negotiable, you break the system. The portrait describes everything, but it tells you of each thing which layer it belongs to.

The recovery process, in the agentic method, divides the work according to the boundary of chapter five. The agent finds. It traverses the system, or better, it traverses the projections of the system, the maps that distil its dimensions, and extracts candidates, this structure looks like an aggregate, this condition looks like an invariant, this datum looks like the identity of this entity. Each candidate comes with its provenance, I extracted it from the behaviour, I inferred it from the structure, the name suggests it. The architect ratifies. They look at the candidates, confront them with the oracle when there is doubt, confirm or reject them, and above all decide what to do with whatever has no clear source.

There is an idea about ratification worth highlighting because it corrects a common misunderstanding. Ratification is not where uncertainty gets resolved. It is where uncertainty is forbidden from entering. It sounds the same but it is not. Resolving uncertainty sounds like the ratification meeting discusses the doubts and reaches agreement about the doubtful. Wrong. The doubtful is not ratified, it is marked as doubtful and sent to be verified against the oracle or investigated. What is ratified is what is already solid, what has a firm source. Ratification is a filter that lets through only what can defend its provenance, not a tribunal that turns conjectures into facts by consensus. If you leave ratification having blessed as certain something that entered as doubtful, with no new evidence, you have corrupted the model with the authority of a signature.

An honest warning to close. Recovery is never complete. There will always be behaviour in paths that are not exercised, cases the oracle has not seen because it was not asked, dormant rules that only wake in rare circumstances. That is irreducible residue, and the right attitude is not to pretend it does not exist but to mark it. The honest recovered model includes its own gaps, the zones where it knows it does not know. Those gaps are not a failure of the recovery, they are its most valuable part, because they tell you exactly where the new system can surprise you, and therefore where to watch. A model that pretends to be complete lies. One that marks its gaps tells you where to look.

With the domain recovered, portrayed, marked and with its gaps flagged, the elevator goes up. It is time to decide the architecture of the new system. And there, at last, the architect chooses.

9. From the Recovered Domain to the Target Architecture

The moment translation skips and that gives meaning to everything before arrives. You have a model of the domain, faithful, marked, with the semantics separated from the old structure. Now you decide what the new system will be like. This is design, in the full sense, and therefore it is the architect's territory, not the agent's. The agent sets the table of options, as we saw. The architect chooses the dish.

The first liberating idea is the one we already planted in chapter three. **Over one recovered domain many architectures fit.** The domain is one. The ways of structuring it are many. You can group the contexts one way or another, you can choose one architectural style or another, you can decide which parts deserve model richness and which can stay simple. None of those decisions is dictated by the domain. They are all the architect's judgements about what best serves today's constraints, maintainability, performance, team autonomy, whatever matters in your case. This multiplicity is exactly the prize of having separated the semantics from the structure. If you had kept the old structure, there would be no choice. Because you have let it go, there is a fan.

The design work divides into two halves of very different nature, and separating them avoids much confusion. There is a mechanical half and a half of choice.

The mechanical half is the translation of patterns of the recovered domain to patterns of the target architecture, when that translation is direct. A domain operation becomes a unit of execution of the new architecture. A domain event becomes a message. A batch process becomes a stream. These correspondences are fairly mechanical once the architectural style is decided, and they are just the kind of work the agent does well and fast. Here the concrete frameworks enter as examples. A framework that organizes code into vertical slices, each with its explicit business intent, is an example of a target architecture that fits naturally with a domain recovered in terms of operations and rules, because the vertical slice preserves the cohesion of each operation. But the framework is the example. The durable concept is that there is a part of the translation to the new architecture that is mechanical and delegable.

The half of choice is where the architect lives. Here are the decisions the domain does not dictate. Do you turn a concept the old system had anaemic, a bag of data with no behaviour, into a rich aggregate that protects its own invariants? Do you redraw a boundary the old system had badly placed? Do you turn a loose datum into a value object with its own meaning? Each of these is a design choice, and each has consequences, not only on the quality of the new system but, crucially, on how expensive the later verification will be. An architecture more faithful to the old struc-

ture is cheaper to verify, because it resembles the oracle more. A more ambitious architecture, one that redraws boundaries, is more expensive to verify, because it diverges more. The agent can and should quantify that consequence for each option. The architect decides with that information on the table.

There is a recurring decision that illustrates well why human judgement is irreplaceable here, and it is the relocation of an invariant. In the old system, a rule could be enforced in several places at once, repeated, because several parts of the system wrote the same datum and each checked the rule on its own. On redesign, the natural thing is to relocate that rule to a single place, the aggregate that owns the datum, so that it protects it once and well. Mechanical, in appearance. But there is a trap that only judgement detects. Relocating the rule to a single place assumes that all the old copies of the rule were identical. What if they were not? What if one of the parts that wrote the datum applied the rule and another did not, deliberately, because in that concrete flow the rule should not apply? Then consolidating the copies into one is not a cleanup, it is a change of behaviour, and possibly a serious error. Detecting that divergence requires looking at the recovered domain with a critical eye, and deciding whether the copies were the same rule badly repeated or distinct rules that looked alike. That is judgement. The agent can flag that there are several copies and show you whether they differ. Deciding what that difference means and what to do with it is the architect's, and it is exactly the kind of decision that, if you delegate it, blows up on you months later.

There is a prudent order for all this, and it connects again with 3X. Do not chase the ambitious architecture on the first attempt. Start with a conservative design, as faithful as possible to the recovered structure while it still makes sense, even if it is anaemic and inelegant. That conservative design is cheap to verify, because it resembles the oracle, so you close the equivalence fast and you have a new system that demonstrably does the same as the old. Only then, with the safety net in place, do you tackle the ambitious improvements, enriching models, redrawing boundaries, as later steps, each with its own verification. The domain specification, which we will see in the next part, gates both the conservative and the ambitious design alike, without changing, because it is expressed in terms of the domain and not of any concrete architecture. First prove you break nothing with a humble design. Then improve, step by step, never letting go of the net. Optimizing before you have the net is, again, exploring where it was time to stabilize.

This leads us straight to the pillar that makes everything before safe rather than reckless. You have recovered a domain and decided a new architecture that diverges from the old. How do you prove, not assume, prove, that the new system means the same as the old? That is Part IV, and it is where the method stops being a good intention and becomes a verifiable commitment.

Part IV. Proving You Did Not Break It

10. The Transitive Chain

Here is the problem in its crudest form. You have the old system, which works but whose design you abhor. You want a new system, with a design you have chosen and that diverges on purpose from the old. And you have to prove that the new behaves the same as the old in everything that matters. The temptation is obvious. Compare the new with the old, directly. Same input to both, compare the outputs, if they match, equivalent.

That direct comparison is harder than it looks, and sometimes impossible to do well, for a reason that comes from chapter three. The new system and the old diverge inside on purpose. They have distinct boundaries, distinct data structures, perhaps they even speak distinct languages at their interfaces, because the old speaks in terms of its old mechanism and the new in terms of its clean domain. Comparing two systems that do not even present the same shape of input and output requires a layer of translation between them, and that translation layer is new, unverified code, interposing right where you want certainty. You end up entrusting your proof of equivalence to a translator whose correctness nobody has demonstrated. The direct comparison puts an unknown at the centre of the proof.

The solution is a detour that turns out to be straighter than the direct line. **Never compare the new system against the old directly. Build a transitive chain of equivalences.** The idea is to introduce intermediate steps chosen so that each link of the chain is easy to verify, and let transitivity do the rest. If A is equivalent to B, and B is equivalent to C, then A is equivalent to C, without ever having compared A with C directly.

Let us look at the chain. The first link is to build a replica of the old system that preserves its structure, not its design, its structure. This is exactly the oracle of chapter two, the copy that runs fast and local. Because it preserves the structure of the old, proving that replica equivalent to the original old system is easy, because they are nearly identical inside, so the comparison is almost an identity and can be done with a fine-grained verification, comparing even internal details, because the structures correspond. This link is cheap precisely because there is no design divergence to bridge. Once it is closed, you have a faithful replica of the old behaviour that lives on your machine and answers in milliseconds. And, importantly, from here the original old system can be retired from the equation of daily work, because its behaviour already lives, faithfully, in the replica.

The second link does not compare systems, it captures behaviour. You use the faithful replica as an oracle and observe, systematically, what it does over a well-chosen set of inputs. You write the result of that observation as an executable specification, a

collection of cases that say, given this situation and this request, the observable behaviour is this. That specification is the next link of the chain, and it has a magic property. It is expressed in terms of the domain, not in terms of the structure of any system. It does not say what the old system does inside nor what the new will do inside. It says what is observed, in domain language. That is why it can serve as a pattern for both.

The third link is the new system. You build it, with its divergent design, and you prove that it satisfies the captured specification. You do not compare it with the old. You compare it with the specification, which is in domain language and therefore does not require a dubious translation layer, it only requires that the new system, expressed in its own clean domain, produces the observables the specification dictates.

And there transitivity closes. The old system is equivalent to its faithful replica, first link, fine-grained verification. The faithful replica produces the specification, second link, by direct observation. The new system satisfies the specification, third link. Therefore the new system is equivalent to the old, without ever having compared them directly, without ever having built that dubious translation layer between two systems that diverge inside. The specification, in domain language, is the pivot that connects the two worlds without touching them to each other.

There is an additional elegance in this chain worth noting. The granularity of the verification follows the structural resemblance of each link, and this is a general principle worth remembering. Where two things resemble each other a lot inside, first link, old against replica, you can and should verify finely, looking at internal details, because the correspondence exists. Where two things diverge inside on purpose, third link, specification against new system, you cannot look at internal details because they do not correspond, so you verify only what is observable from the outside. The granularity of the proof adjusts to how much the things you compare resemble each other. Forcing a fine comparison between things that diverge is the error the transitive chain avoids.

It remains to develop what exactly that captured specification is, how it is written so as to be at once a proof pattern, a regression net and documentation, and why its independence from the languages of the two systems is what protects your freedom to redesign. That is the next chapter.

11. Behaviour as an Executable Specification

The captured specification is the pivot of the whole verification, so it deserves a whole chapter on what it is and why it has the shape it has. The central idea is that the observed behaviour of the oracle is written as an executable specification, a collection of cases a machine can run and check, not a prose document someone has to interpret.

The most useful shape I know for this is the tabulated example, in the style of given this, when that, then this other. Each case describes a starting state, given this, an action, when that, and the expected observable, then this other. A row of the table is a case. The whole table is the specification of one domain operation. This shape has an enormous pedagogical virtue, because it reads almost like natural language and can be reviewed by a business expert who does not program, and at the same time it is executable, because each row can be run against a system and you can check whether the real observable matches the expected one.

The most important thing about the specification, and what makes it the pivot of the transitive chain, is the language it is written in. **The specification is written in the ubiquitous language of the domain, not in the dialect of either of the two systems.** The old system speaks in terms of its mechanism, its memory structures, its technical codes. The new system speaks in terms of its clean domain. If you wrote the specification in the old one's dialect, you would chain the new system to the old shape and lose the freedom to redesign that cost you so much to win. If you wrote it in the new one's dialect, you could not use it to verify against the old. The only shape that serves both is a third language, that of the pure domain, independent of both implementations. That is the language in which the business expert would describe the rule knowing nothing about computers.

This has a concrete architectural consequence. Because the specification is in domain language and each system speaks its own dialect, a translation piece per system is needed, taking a case in domain language and running it against that concrete system, translating the input into the system's dialect and the output back into the domain. There is one of these pieces for the old and one for the new. The virtue of isolating the translation in these pieces is that the dependence on each system's dialect is confined there, in a small, well-bounded place, instead of contaminating the specification. The specification stays pure, in domain, reusable, and the dirt of each concrete dialect lives isolated in its translation piece. That purity is what protects your freedom to redesign, because you can change the whole new system, its dialect included, and the specification still holds, you only change its translation piece.

The specification captures the complete domain observable, and you have to be careful about what complete means. It means not only the direct answer to the request, but also the side effects that matter to the domain, the state change the operation leaves behind. If an operation, besides answering, modifies the state of an entity, that state change is part of the observable and must be in the specification. But, and this is key, those side effects are observed through domain queries, asking the system about its state in domain terms, not spying on its data store inside. Spying on the store would break the black box and again tie the proof to the concrete storage structure, forbidding the redesign. You ask about the state in domain language, just as you asked about the answer.

There is an honest limit worth declaring. Complete means all the domain observable that matters, not literally every effect. Purely technical effects, an entry in a log, an internal counter, a status code of the plumbing, are not captured, because they are not domain, they are mechanism, and comparing them would be comparing exactly what you have the right to change. And there is a deeper limit, you can only capture what the system exposes. If there is a domain effect that no query brings to light, it stays out of the specification, it is a blind spot. Capturing all the observable is the goal, but it requires the system to expose, via domain queries, all its domain effects, and that does not come for free, it has to be designed.

To finish, an observation about why this specification is so profitable. Each row of the table lives three lives at once. While the old system or its replica are still alive, the row is a parity test case, it checks that old and new coincide. When the old one retires, the same row becomes a regression test of the new system, it checks that no future change breaks the inherited behaviour. And always, from the first day, the row is living documentation of the domain, in language the business understands, that does not lie because it is executable and would break if it lied. Few things in engineering give three uses for the price of one. The domain specification is one of them, and that is why it is worth the effort of capturing it well.

One last piece is missing for the verification to be solid, and it is the most subtle of all. What exactly you compare when you compare the observable, and in what order you do things. That is the last chapter of this part.

12. Black-Box Parity and the Law of Order

We already know we compare what is observable from the outside, not the innards. We call that way of verifying black-box parity, because it treats each system as a closed box of which only the inputs and outputs are seen. But black box does not mean comparing blindly everything that comes out. It means choosing carefully which of what comes out counts as a domain observable and which is technical noise to be ignored. That choice, what gets compared, is the most delicate dial in the whole verification, and miscalibrating it ruins the proof in one of two opposite directions.

If you compare too much, including in the comparison technical details the new system has the right to do differently, the proof fails constantly over differences that do not matter, a distinct internal code, a distinct date format, a distinct field order. You drown in false positives, the proof becomes noise, and you end up ignoring it, which is the worst thing that can happen to a test. If you compare too little, excluding things that were domain observables, the proof passes even though the behaviour has really changed, and it gives you false confidence, which is even worse, because it lets you break the system with a clear conscience. Between comparing too much and comparing too little there is a right calibration, and finding it is the craft.

The calibration rule is this. Classify each field of the observable into one of several categories. The domain results and the resulting domain state are strict comparison, they have to match exactly, because they are the semantics you promised to preserve. The purely technical fields, mechanism with no domain consequence, are excluded, because they are exactly what you have the right to change. The non-deterministic fields, a generated identifier, a timestamp, are normalized before comparing, reduced to a canonical form that ignores what legitimately varies. And there is a dangerous category, the ambiguous or entangled fields, where a technical value hides a domain consequence, and those must be untangled carefully, comparing the domain consequence and not the technical value that carries it.

The golden rule of this classification is to compare strict by default and exclude only with evidence. The burden of proof falls on exclusion, not on inclusion. If you doubt whether a field matters, you compare it, because the risk of comparing too much is annoying noise, while the risk of excluding too much is dangerous blindness. You only exclude a field when you can prove it is technical and that it drags no domain consequence. And that proof should be traceable, every excluded field traces to a concrete piece of mechanism with no domain effect. Excluding for comfort, because the field is annoying, is opening the door to blindness.

The case of errors illustrates the subtlety well. When an operation fails, do you compare the failure mode or the domain outcome? The right answer almost always is the

domain outcome. If the old system rejects an operation by a business rule, and the new one rejects it by the same business rule, they are equivalent, even though inside the mechanism of the rejection is completely different. What matters is that the operation was rejected for that domain reason, not how the rejection was implemented. By contrast, if the old one fails by a technical breakdown, that is not domain, it is mechanism, and it should not be part of the comparison. Distinguishing the business rejection from the technical failure is exactly the untangling of chapter three, where we warned that the silent loss of semantics hides there. Well-calibrated parity is where that distinction becomes operational.

The second theme of the chapter is missing, and it is a law of order that has the rank of law in this method. **Equivalence is closed before any quality refactor begins.** That is, first you prove the new system equivalent to the old, with its initial conservative design, and only then, with that equivalence proven and green, do you start improving the internal quality of the new system, refactoring, enriching, polishing. Not the other way round. Never the other way round.

The reason is structural, not a matter of preference. Refactoring is changing the structure while preserving the behaviour. To do it safely you need a net that tells you whether you have broken the behaviour while you changed the structure. That net is precisely the equivalence specification. If you refactor before having the equivalence closed, you refactor without a net, and a change that breaks the behaviour goes unnoticed, because you have nothing to detect it. Worse still, if you do both things at once, redesigning for equivalence and refactoring for quality mixed together, when something fails you will not know whether it failed because your redesign was not equivalent or because your refactor broke something. You will have lost the ability to diagnose, which is the same sin as the double jump of chapter one, making two distinct changes in the same move.

This law of order has an authority worth embodying in something, a guardian that enforces it. Call it what you like, what matters is its function. It is the equivalence authority. It owns the specification, which is its evidence. It runs the comparisons. It custodies the green runs, with their coverage and their provenance, so the record shows what has been proven and how strongly. And it enforces the order, it does not let the quality refactor begin until the equivalence is closed. It is not just a tool that generates tests. It is the referee that guarantees the method is followed in the right order, because the history of modernization disasters is full of teams that refactored on sand, on code they believed equivalent without having proven it, and that discovered in production that it was not.

With the whole verification in place, transitive chain, domain specification as pivot, well-calibrated black-box parity, and the law of order guarded, you have the safety

net that makes the redesign engineering and not a gamble. But there is still the longest and least glamorous part of any real modernization. The old system does not switch off the day the new one is ready. The two coexist, sometimes for years. Coexisting well is an art of its own, and it is Part V.

Part V. Living with Two Systems

13. The Strangler, Honestly

Almost everything written about modernization treats the change from old to new as an instant, the day of the switch, as if there were a switch. In the reality of any system that matters, there is no switch. There is a period, often long, in which the old system and the new coexist, serving the operation of the business between them, while the new gradually takes on responsibilities. Designing that coexistence is first-class architecture work, not a deployment detail, and underestimating it is one of the most common causes of a technically correct modernization ending in operational disaster.

The classic pattern for this is the strangler, which Martin Fowler named after a fig that grows around a tree until it replaces it. The idea is to put a facade in front of everything, that receives all requests, and that for each one decides whether the old system or the new one serves it. At first almost everything goes to the old. Gradually, as each part of the new is proven equivalent, the facade starts routing that part to the new. The fig grows. The old tree shrinks. One day the old serves nothing and can be retired.

Described that way it sounds clean, and the routing part really is. The facade that decides whom to send each request to is, conceptually, simple. The problem, and here comes the honesty of the title, is that routing is the easy and visible part, while the hard and invisible part is underneath, in the data. And almost all the literature gets excited about the easy part and stays silent on the hard one.

Let us see why the data are the problem. While the two systems coexist, there is a piece of business data, say the state of a central entity, that some operations the new system already serves and others the old one still does. If an operation that has already migrated writes that data into the new store, and another that has not migrated writes it into the old store, the data has split into two. There are two versions, in two stores, diverging. The new system believes one thing about the entity, the old one believes another, and the business is corrupted without anyone noticing until it is too late. The facade that routes operations does not solve this at all, because the problem is not in the operations, it is in the state they share.

From here comes the principle that governs the carving, and it is less obvious than it seems. **Carve along write-ownership, not where it is convenient for you.** That is, group together for migration all the operations that write a given piece of data. If four distinct operations write the state of the central entity, those four migrate at once, in the same step, or none migrates. If you migrate two and leave two in the old, you split the state and corrupt it. The boundary you carve along has to respect who owns the writing of each piece of data, because only that way do you guarantee that

at each moment a piece of data is written by a single system, and therefore cannot diverge.

This explains a carving order that would otherwise seem arbitrary. The parts of the system that are peripheral, that read a lot and write little or write data that only they touch, are easy to migrate first, because carving them off splits no shared state. The central parts, where several operations fight to write the same critical state, are the ones to migrate last and in a block, because they are entangled by write-ownership. The core migrates whole or it does not migrate, and it is left for last precisely because it is the knot. Whoever tries to migrate the core in little pieces, one operation today and another next month, is splitting the most critical state of the business at every step. It is the shortest road to silent disaster.

The facade, moreover, does double duty if you design it well. The same translation piece that in the verification converted between the domain language and each system's dialect serves here as a bridge between the two systems during coexistence. It is not a coincidence. It is the economics of the method rewarding coherence. A piece well designed to isolate the dependence on a dialect serves equally for verifying and for coexisting. We will see it again in the next chapter, where coexistence becomes the last test before the definitive change.

For now, the lesson is sober. The strangler is an excellent pattern, but its famous part, routing, is the easy one. The part that decides whether your modernization lives or dies is data coexistence, and it is governed by carving along write-ownership so that no piece of data has two owners at once. If you internalize only one thing from this chapter, let it be that.

14. Cutover Without Faith

The moment to really change a part of the system arrives, to stop routing it to the old and route it to the new definitively. That moment is called cutover, the cut. And the question is, when is it safe to do. The lazy answer, when the tests pass, is insufficient and dangerous, because the tests passing once in your environment does not tell you how the new system will behave against real traffic, with all its variety and its rare cases that your tests perhaps do not cover.

The method proposes a cutover criterion that does not rest on faith but on accumulated evidence, and it has several conditions that must hold at once. The first, the equivalence specification is green, the new system coincides with the oracle in all the strict domain observable. The second, the coverage of that specification is sufficient, it has exercised the branches, the boundaries, the limit cases, not only the happy path. A green specification with poor coverage is a false reassurance, because it only proves they coincide where you looked, and where you did not look the bugs live. The third condition is the decisive one and the one people in a hurry most often skip.

The third condition is to run the new system in shadow over real traffic, for a period, and check that it coincides with the old. Running in shadow means the real production traffic goes to both systems at once, but only the old one's answer counts, the one still in charge. The new one processes the same traffic in parallel, its answer is compared with the old one's, but it is not used, it does not affect the business. It is a rehearsal with a net, the new one acting on reality without consequences. During that period you accumulate evidence that the new coincides with the old not over the cases you happened to think of, but over the real distribution of what actually happens, including the rare cases that neither your specification nor your imagination had covered. Shadow is where the silent loss of semantics that escaped capture surfaces, because real traffic is the most exhaustive examiner there is, it tests everything, without mercy and without warning.

The fourth condition closes the criterion. Every divergence observed in shadow has been investigated and resolved, or ratified as acceptable. A shadow divergence is gold, it is the system warning you of a case where new and old do not coincide. Each one deserves investigation. Almost always it reveals a fault of the new one that you fix. Sometimes it reveals that the divergence is desired, a rule change you decided on purpose, and then it is ratified and documented as such. What is never done is to ignore it. Cutover with unexplained shadow divergences is cutover with faith, exactly what the method rejects.

There is a historical lesson behind all this caution, and it is worth naming even in the abstract. The modernizations that have ended in disaster headlines, systems that cor-

rupted data massively after the change, almost always share the same root. It was not a tool failure. It was a governance failure. Someone asserted equivalence instead of proving it, cut over on that assertion, and reality collected the difference. The cutover-without-faith criterion, green gate plus coverage plus shadow plus resolved divergences, is the direct answer to that class of disaster. You do not cut over on what you believe. You cut over on what you have proven and confirmed against reality.

When the four conditions hold, the cut can be made, and it is worth making gradual even then. Instead of sending all the traffic to the new at once, you send it a small fraction as the system in charge, you watch, and if it holds, you raise the fraction little by little. It is prudence taken to the last metre. Even with all the evidence accumulated, a gradual change leaves you an exit if something unexpected appears with the new system really in charge.

An honest technical warning so as not to sell shadow as free. Running in shadow requires the new system to process real traffic without causing real effects on the world. Its writes have to go to a shadow store, not the production one. Its external effects, sending a notification, calling a third party, have to be suppressed, because otherwise the rehearsal becomes reality. For operations of pure computation and internal state change, this is clean. For operations with visible external effects, it is more delicate, you cannot really send the notification twice, so you suppress the external effect and compare only the internal decision that would have fired it. Designing that suppression well is part of the cost of shadow, and it is worth budgeting for it instead of discovering it late.

With the cutover done, part by part, the new system takes over and the old is left without work. The last question of coexistence arrives, and it is subtler than it seems. How the old system is retired without losing what only it knew.

15. Retiring the Oracle Without Losing the Reference

The old system, and its faithful replica that has played the oracle, have been your source of truth throughout the modernization. Every time you doubted what the new system should do, you asked the oracle. Now the new one is in charge and the old one is redundant. The temptation is to switch it off and celebrate. But switching off the oracle is losing forever the only source that knew, with certainty, what the old system did in each case, including the ones you never captured. It has to be retired with care, because it is a care that cannot be repeated, once switched off, there is no going back.

The sequencing rule is blunt. **Capture everything you will need from the oracle before retiring it, because afterwards you will not be able to re-derive it.** While the oracle lives, you can ask it any new question that occurs to you and get the truth. When it dies, the only reference that remains is what you captured, the frozen specification. If a case is not in the specification and the oracle no longer exists, that behaviour has been lost as a source of truth, you only have what the new system does, which you can no longer contrast against anything. That is why retiring the oracle is gated by a condition, the specification captures everything we are going to need, to the coverage we have agreed is sufficient. While that condition does not hold, the oracle is not switched off.

Once everything is captured and the oracle retired, the frozen specification becomes the permanent reference, and here you see why we insisted so much on its shape. The specification, which was a parity pattern while the oracle lived, becomes the regression net of the new system, now only with its translation piece, without the old one's which no longer exists. Every future change of the new system is checked against that frozen specification, which guarantees the inherited behaviour is preserved. And if you ever have to deliberately change an inherited rule, that change is made by editing concrete rows of the specification, explicitly and ratified, not by accident. The specification is the memory of the old system, written so that it survives its author.

There is a practical cost nuance worth taking advantage of, and it distinguishes two retirements people confuse. One thing is retiring the old system for real, the original, the one that ran on the expensive platform, and that is expensive and is done at the definitive cutover, freeing the cost that justified the whole modernization. Another thing, very different, is retiring the faithful replica, the local oracle, which runs cheaply on your own infrastructure. The replica is cheap to keep switched on. So a prudent strategy is to retire the original old system as soon as the cutover allows, to capture the saving, but keep the faithful replica a while longer, as a backup oracle. If after the cutover a surprise appears in production, a case the new system handles du-

biously, you can ask the replica again what the old one would have done, and disambiguate. The replica is the cheap insurance you keep after switching off the expensive thing, and you retire it too when the accumulated confidence makes it no longer worthwhile.

It is time to close with the honest residue, because this method does not promise absolute certainty and pretending it does would betray everything said. When the oracle, original and replica, no longer exists, the specification is the only reference, and the specification is finite. All behaviour of the old system that was not captured, the paths that were never exercised, the rare cases that not even the shadow traffic fired, has become irrecoverable from its original source. If any of those cases wakes one day in production, there is no longer an oracle to ask what was correct, all that remains is to decide it anew, now as design, not as recovery. This is not a failure of the method, it is a limit of reality, no process can capture what never manifested. What the method does is bound that residue and make it visible, capture broadly while the oracle lives, run the shadow over real traffic to drag the rare cases into the light, and honestly mark the gaps that remain. After retiring the oracle, reality is the only teacher left. The method makes sure you reach that point having learned from the oracle everything that was there to be learned.

Part VI. From Explore to Expand

16. This Is a Hypothesis

Having reached this point, it is worth taking off a mask that many technical books never take off. What you have just read is not a method proven against hundreds of production migrations. It is a hypothesis, coherent and reasoned, about how modernization should be done when you take seriously that it is a domain problem and you take advantage of the fact that there are now agents on every floor of the elevator. It is, in Beck's terms, in the Explore phase. And saying so does not weaken the book, it strengthens it, because an argument presented as what it is endures better than a promise presented as what it is not.

It is worth gathering why the method should work, because that is the substance of the hypothesis. It should work because it attacks the real cause of the failure, confusing modernization with translation, instead of a symptom. It should work because it rests on a solid source of truth, the observable behaviour of the living system, instead of documentation that lies or code that deceives. It should work because it preserves what truly matters, the semantics, and frees what truly gets in the way, the old structure, instead of confusing the two things. It should work because it proves equivalence instead of assuming it, and it proves it by a chain of steps each easy to verify. And it should work because it treats coexistence and the change with the respect they deserve, instead of pretending a switch exists. Each of those pieces rests on principles older than the method, DDD, the oracle, verification by transitivity, the strangler, which have survived their own test of time. The bet of the book is that combining them this way, with the domain at the centre and the agent as augmentation on every floor, produces something more than the sum, a process that earns the word modernization.

But a hypothesis is not a conclusion, and it is worth being clear about what would turn it into something more. Moving from Explore to Expand requires evidence that right now does not exist. It requires walking the whole method over real production engagements, several, distinct from one another, and seeing not only that it works but understanding why it works well enough to predict when it will not. That capacity to predict failure is Beck's sign of maturity, the boundary between a lucky hunch and a method. Until you have it, the honest attitude is to keep exploring, walk the elevator in fast cycles, learn from each one, discard without grief whatever reveals itself wrong, and resist the temptation to freeze and productize too soon.

There are open questions the method itself does not resolve and that the Expand phase will have to address. How does domain recovery scale when the system is not one but a constellation of systems entangled with one another? What happens when the recovered domain reveals that the business itself no longer wants to do what the old system did, when modernization uncovers an opportunity to change the rules and

not only preserve them? Where is the real limit of what an agent can recover reliably before human judgement has to carry almost everything? How is all this governed when there is not one architect but many, with distinct criteria, over the same system? None of these questions has a closed answer here, and pretending it does would be putting the mask back on. They are the work that remains.

That honesty is, in the end, coherent with the method itself. The whole book preaches that it is better to prove than to assume, that provenance matters, that what is not known has to be marked instead of pretending certainty. It would be incoherent for the book to exempt itself from its own discipline and present itself as proven truth when it is a reasoned hypothesis. So here is its provenance, marked as the method demands. This is an argument from principles, not a validated field finding. Its strength is in the soundness of the reasoning and the quality of the principles it combines, not in evidence that does not yet exist.

If the method is good, the walking will prove it, engagement by engagement, cycle by cycle, in the Explore phase where it now lives, until the evidence justifies scaling it. And if in that walking it reveals itself wrong in some part, the right attitude will not be to defend it, it will be to learn from the failure and redo it, because that is exactly what exploring means. This book is not the end of an investigation. It is the best shape I have managed to give, today, to a question that remains open. How you really modernize a system that matters, putting the domain at the centre, with an agent on every floor of the elevator, and without breaking anything along the way. The invitation is to walk it, not to believe it.

Appendices

Appendix A. Antipatterns

Antipatterns are the errors the method fights, gathered here so you can recognize them quickly. Each one is the shadow of a principle in the book.

The translation that believes it is modernization. Transpiling the old system into the new language line by line and declaring the work done. It preserves the old design and the interwoven mechanism, now in modern syntax and therefore harder to detect. It is the root antipattern, the one that gives meaning to the whole book.

The recompilation that believes it is the destination. Running the old system on a new platform without touching its design, and believing that with that you have modernized. It is a legitimate relocation disguised as a transformation. The error is not recompiling, it is believing you are finished.

Refactor first, the net later. Starting to improve the structure of the new system before having proven its equivalence with the old. Refactoring without a net, with nothing to detect whether you have broken the behaviour. It violates the law of order, and it is the root of the most notorious modernization disasters.

The double jump. Changing the platform and the design in the same move. When something fails, and something fails, you cannot diagnose whether it was the platform change or the design change. You lose the ability to isolate the cause just when you most need it.

The choice disguised as a finding. Slipping a design decision into the recovered model as if it were something observed in the old system. It contaminates the portrait of the domain, because it mixes what the system does with what you want it to do. The discipline of provenance exists to detect it.

The ratification that resolves doubts. Turning, in a ratification meeting, a doubtful conjecture into an accepted fact, by consensus and with no new evidence. Ratification must filter what has a firm source, not bless the doubtful with a signature.

The silent loss of semantics. Discarding as technical mechanism something that hid a business rule. The typical case is the handling of a technical error that in fact encoded a domain decision. Where mechanism and logic interweave, there this antipattern lurks, and only the oracle disambiguates it.

The parity that compares too much. Including in the equivalence comparison technical details the new system has the right to do differently. It produces a sea of false positives, the proof becomes noise, and the team ends up ignoring it, which is the worst thing that can happen to a test.

The parity that compares too little. Excluding from the comparison things that were domain observables. It produces false confidence, lets real behaviour changes through, and invites you to break the system with a clear conscience. Worse than the previous one, because silence deceives more than noise.

The carving that splits the state. Migrating in little pieces a set of operations that write the same data, leaving some in the old system and others in the new. The data splits into two diverging stores. It corrupts the most critical state of the business without warning. It is avoided by carving along write-ownership.

The cutover with faith. Cutting over to the new system because the tests pass, without having run in shadow over real traffic nor having resolved the divergences. It is asserting equivalence instead of proving it against reality. The history of modernization disasters is written in large part with this antipattern.

Switching off the oracle too soon. Retiring the old system and its replica before having captured everything you were going to need. Once switched off, there is no going back, and the uncaptured behaviour is lost as a source of truth forever.

Productizing in Explore. Taking a method or an approach that worked once and freezing it as a corporate standard before understanding it well enough to predict when it will fail. It jumps from Explore to Extract without passing through Expand. One success is not a pattern.

Appendix B. Glossary

Agent. A system capable of reading a great deal of material, reasoning about it and producing results. In this book, an augmentation of the architect present on every floor of the elevator, that finds but does not choose.

Architect's elevator. Gregor Hohpe's metaphor. The architect goes up and down between the penthouse, where strategy is decided, and the engine room, where code executes, connecting floors that would otherwise not speak to each other.

Transitive chain. A verification strategy that avoids comparing the new system with the old directly. It proves easy intermediate equivalences, old with faithful replica, replica with specification, specification with new system, and lets transitivity conclude the final equivalence.

Cutover. The cut. The moment when a part of the system stops being served by the old and passes to being served by the new definitively.

Double jump. The antipattern of changing platform and design in the same move. Forbidden because it destroys the ability to diagnose failures.

Executable specification. A collection of cases in the form of given this, when that, then this other, written in domain language, that a machine can run and check. It is the pivot of the transitive chain and lives three lives, parity, regression and documentation.

Fidelity. The commitment to preserve the invariants and the semantics of the old system, not its structure. What the system means is sacred, how it achieves it is free.

Invariant. A rule the domain must always satisfy. Part of the problem space, it survives any redesign.

Ubiquitous language. The shared vocabulary of the domain, in which the specification is expressed, independent of any concrete system's dialect. Its independence is what protects the freedom to redesign.

Mechanism. The plumbing that makes the logic work on a concrete platform. Old solution space. It is what you have the right to change, as opposed to the business logic, which you must preserve.

Recovered model. A faithful portrait of the old system's domain, in DDD terms, stripped of mechanism, with each statement marked by its provenance. It describes the current state of the domain, not the desired one.

Oracle. The old system, or its faithful replica, as a source of truth about behaviour. You give it an input, it gives you the correct output by definition. It replaces documentation and memory as a source.

Black-box parity. A verification of equivalence that compares only what is observable from the outside, domain inputs and outputs, never the innards, because the innards diverge on purpose.

Provenance. A mark that declares where each statement in the recovered model comes from, and therefore how much you can trust it. Extracted, inferred, attested, unproven. What can cite no source is a choice, not a finding, and leaves the recovered model.

Faithful replica. A copy of the old system that preserves its structure and runs fast locally, made for example by recompiling onto a modern platform. It serves as a cheap, fast oracle. Not to be confused with the new system, which diverges on purpose.

Shadow. A mode of coexistence in which real traffic goes to both systems at once, but only the old one's answer counts. The new one processes in parallel without affecting the business, and its answer is compared. It is the last test against reality before the cutover.

Strangler. Martin Fowler's pattern. A facade in front of everything routes each request to the old or the new, and gradually moves responsibilities to the new, until the old can be retired.

3X. Kent Beck's model. Explore, Expand, Extract. Three phases in the life of an idea, each with its mindset. Confusing them is an expensive error. This method lives in Explore.